ECEN 749 Lab 3 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

Introduction

This lab is to create a custom hardware IP and interfacing it by writing a C code. THe hardware IP that was created for the Zynq processor is a peripheral that can perform integer multiplication. The lab then involves using the SDK in the Vitis IDE to use this multiplication hardware IP.

Procedure

1. The first step is to create the Zynq base system. Once the Zybo board is selected and the hardware target is detected, the next step is to create a block design. The block is to be named multiply. Add the ZYNQ7 processing system IP and re-customize the IP. This is done by importing the TCL file provided and unchecking all the peripheral I/O pins except UART1. The next step is to create and re-package the IP, this is done by selecting AXI4 peripheral and naming it multiply. Even though we need only 3 registers for integer multiplication, the minimum number of allowed registers is 4. Now the peripheral is created which is an AXI lite slave that has 4*32 bit read or write registers. The IP has to now be edited so that 2 of the registers connect to multiplier inputs and another register connects to the output. To edit it, look at the multiply_v1_0 file and click the multiply_v1_0_s00_AXI.v file present. In this file slv_reg2 <= statements needs to be commented out but not the one in the main function. This is because this statement refers to writing to this register, this will ensure that write capabilities of this register will be disabled as it is the one used for multiplier output. In the add user logic part the multiplication logic of slv_reg0*slv_reg1 must be assigned to slv_reg2. After this, Run Connection Automation needs to be clicked. Click on Create HDL Wrapper by right clicking on Sources. Then generate the bitstream and export the hardware to Vitis IDE.
2. In Vitis IDE using the .xsa file that was created by exporting the hardware, this must be used to the new application platform from the hardware. The Hello World file should be modified with the multiplication test application.

Results

1. The macro address has been used to represent XPAR_MULTIPLY_0_S00_AXI_BASEADDR. slv_reg0 is located at the address, slv_reg1 is located at an offset value of 4 adding to the base address, and slv_reg2 is located at the base address plus an offset value of 8. An integer variable called product is declared to read output from the slv_reg2. Two “for” loops with 15 iterations each were used to write to both the registers and the product was read from the 3rd register. The function call MULTIPLY_mWriteReg and MULTIPLY_mReadReg were used for this purpose. Looking at the picocom console we can see that the multiplication IP is working.

2. The macro address, represented by XPAR_MULTIPLY_0_S00_AXI_BASEADDR, defines the location of slv_reg0. Subsequent registers, slv_reg1 and slv_reg2, are located at offsets of 4 and 8, respectively, from the base address. To store the output from slv_reg2, an integer variable named "product" was declared. The process involved two "for" loops, each iterating 15 times, to write data to the registers. The resulting product was then read from the third register. The functions MULTIPLY_mWriteReg and MULTIPLY_mReadReg were utilized for these operations. Observation of the picocom console confirmed the successful operation of the multiplication IP.

Conclusion

This lab taught us how to create a custom hardware multiplication IP and write the C code to test the peripheral. The C code that was used to verify the function of the IP included writing to two registers and then reading them using the m_ReadReg statement. It was also a good learning experience to read the various commands given in the header files and use them to implement the multiplication IP and also know where the registers are located using the base address and the offset and then finally using the picocom console to see the multiplier in action.

Questions

a) The output is a 32 bit integer, it will have a value greater than 32 bits when both the inputs are greater than 16 bits. This error is called overflow. This can be corrected by increasing slv_reg2 register to 64 bits, and keeping the input bits as 32. The “multiply_v1_0_S00_AXI.v verilog code may be modified as follows:

```
Parameter C_S_AXI_DATA_WIDTH = 32;
reg[C S AXI DATA WIDTH-1:0]slv_reg0;
reg[C S AXI DATA WIDTH-1:0]slv_reg1;
reg[2*C S AXI DATA WIDTH-1:0]slv_reg2;
reg[C S AXI DATA WIDTH-1:0]slv_reg3;    //need minimum of 4 registers
```

The width for inputs is 32 bits when establishing the parameters for an AXI4 peripheral [31:0]. For slv_reg2, the register size will be [63:0] because 2*width is used. Increasing the output register size solves this issue.

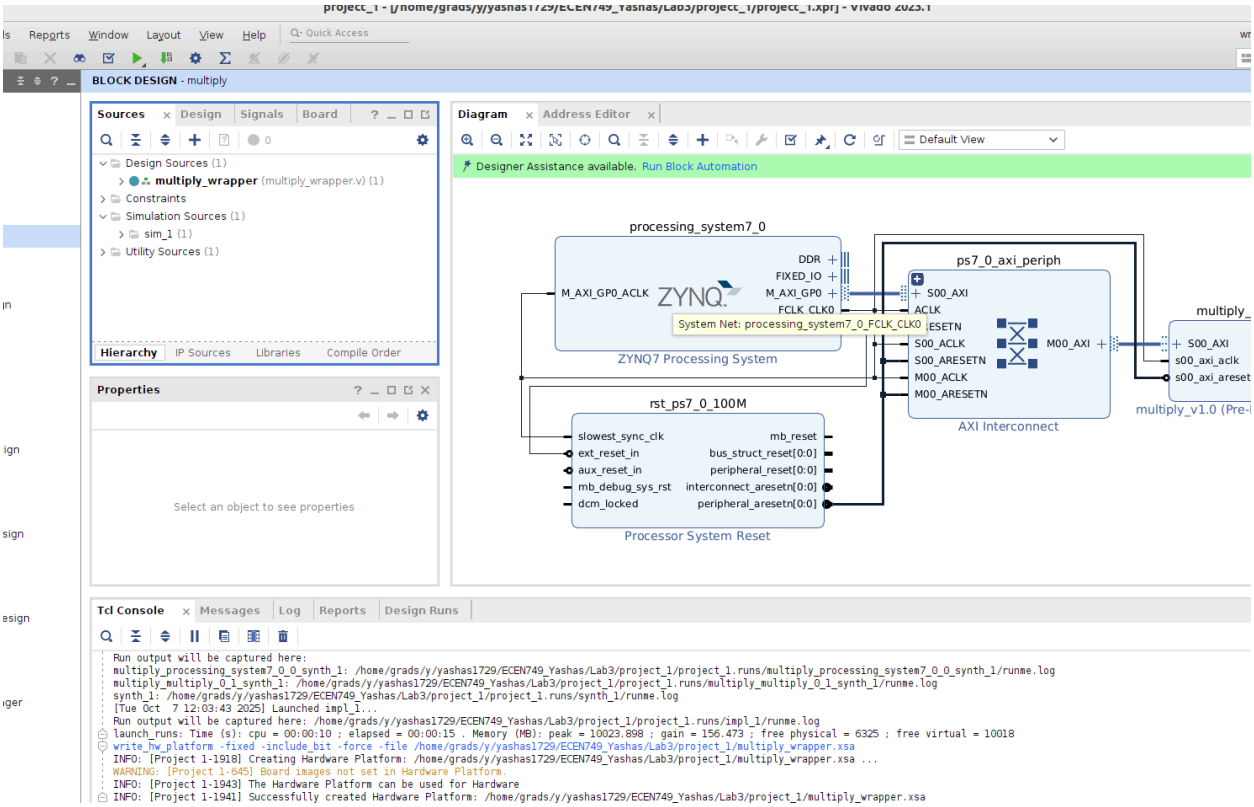
b) If inputs are very long, the multiplier circuit may experience computational delays, potentially causing it to read incorrect data from the output register. If the read command runs before the final output is saved in the output register, we end up getting a garbage or incorrect value instead of the intended result. This situation can be avoided by adding a delay before the read command to ensure that the output is available before the read command runs.

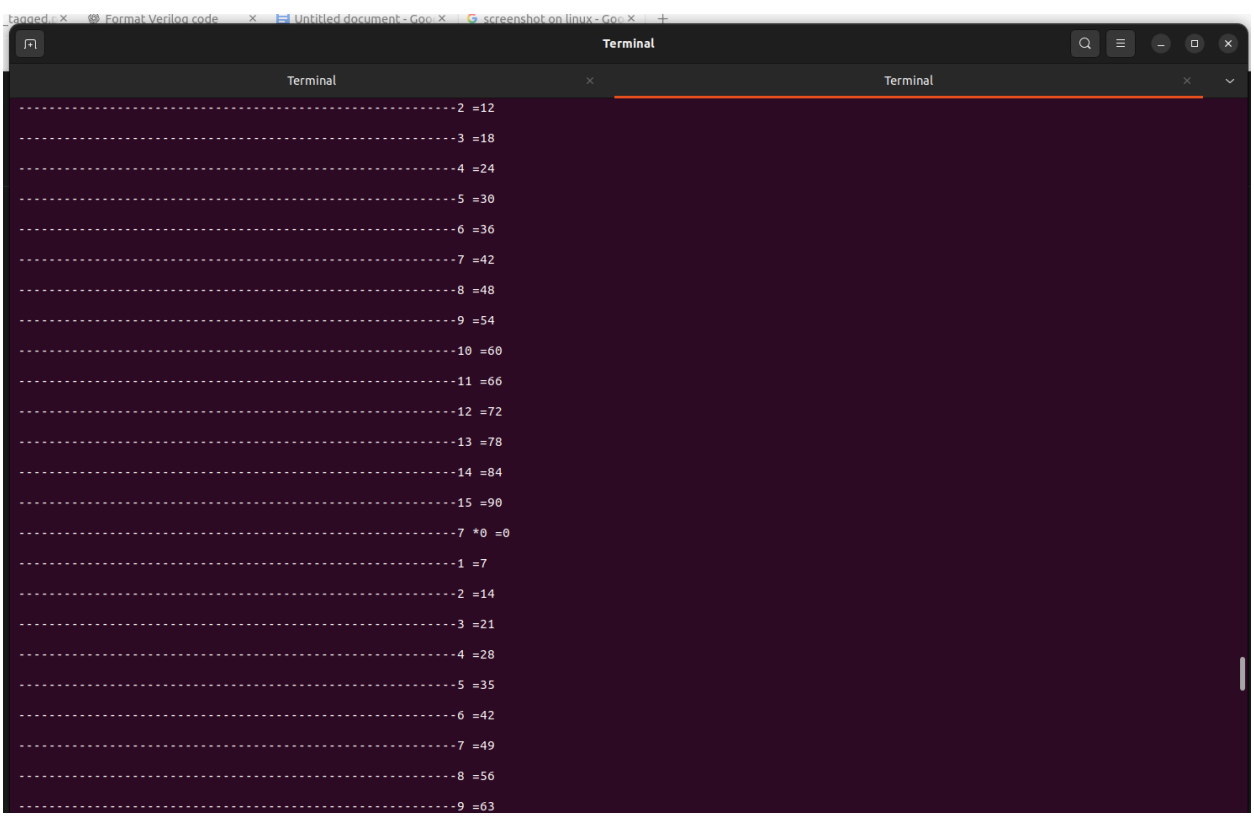
c) An AXI4 peripheral operates in either master or slave interface mode. In master mode, it initiates read/write commands, whereas in slave mode, it responds to these commands. This distinction is crucial for understanding its functionality. Direct testing of a peripheral is not possible when it is configured in master mode.

Appendix

C code:

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "multiply.h"
#include "xil_io.h"

#define address XPAR_MULTIPLY_0_S00_AXI_BASEADDR

int main()
{
    init_platform();
    int product;

    printf("%d *",100);

    for (int i=0; i<16; i++){
        MULTIPLY_mWriteReg(address, 0, i);
        printf("%d *", i);
        for (int j=0; j<16 ; j++){
            MULTIPLY_mWriteReg(address, 4, j);
            printf("%d =", j);

            product = MULTIPLY_mReadReg(address, 8);
            printf("%d %d %d \n",i, j, product);
            printf("\n -------------------------------------------------------");
    }
 }

    cleanup_platform();
    return 0;
}
```


BLOCK DESIGN - multiply
Sources
Design
Signals
Board
Design Sources (1)
multiply_wrapper (multiply_wrapper.v) (1)
Constraints
Simulation Sources (1)
sim_1 (1)
Utility Sources (1)
Hierarchy
IP Sources
Libraries
Compile Order
Properties
Select an object to see properties
Diagram
Address Editor
Default View
Designer Assistance available. Run Block Automation
processing_system7_0
DDR
FIXED_IO
M_AXI_GP0_ACLK
M_AXI_GP0
FCLK_CLK0
ZYNQ
System Net: processing_system7_0_FCLK_CLK0
ZYNQ7 Processing System
ps7_0_axi_periph
S00_AXI
ACLK
ESETN
S00_ACLK
S00_ARESETN
M00_ACLK
M00_ARESETN
M00_AXI
AXI Interconnect
multiply_
S00_AXI
s00_axi_aclk
s00_axi_areset
multiply_v1.0 (Pre-
rst_ps7_0_100M
slowest_sync_clk
ext_reset_in
aux_reset_in
mb_debug_sys_rst
dcm_locked
mb_reset
bus_struct_reset[0:0]
peripheral_reset[0:0]
interconnect_aresetn[0:0]
peripheral_aresetn[0:0]
Processor System Reset
Tcl Console
Messages
Log
Reports
Design Runs
Run output will be captured here:
multiply_processing_system7_0_0_synth_1: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/project_1.runs/multiply_processing_system7_0_0_synth_1/runme.log
multiply_multiply_0_1_synth_1: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/project_1.runs/multiply_multiply_0_1_synth_1/runme.log
synth_1: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/project_1.runs/synth_1/runme.log
[Tue Oct 7 12:03:43 2025] Launched impl_1...
Run output will be captured here: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/project_1.runs/impl_1/runme.log
launch_runs: Time (s): cpu = 00:00:10 ; elapsed = 00:00:15 . Memory (MB): peak = 10023.898 ; gain = 156.473 ; free physical = 6325 ; free virtual = 10018
write_hw_platform -fixed -include_bit -force -file /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/multiply_wrapper.xsa
INFO: [Project 1-1918] Creating Hardware Platform: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/multiply_wrapper.xsa ...
WARNING: [Project 1-645] Board images not set in Hardware Platform.
INFO: [Project 1-1943] The Hardware Platform can be used for Hardware
INFO: [Project 1-1941] Successfully created Hardware Platform: /home/grads/y/yashas1729/ECEN749_Yashas/Lab3/project_1/multiply_wrapper.xsa


Terminal
Terminal
Terminal
----------2 =12
----------3 =18
----------4 =24
----------5 =30
----------6 =36
----------7 =42
----------8 =48
----------9 =54
----------10 =60
----------11 =66
----------12 =72
----------13 =78
----------14 =84
----------15 =90
----------7 *0 =0
----------1 =7
----------2 =14
----------3 =21
----------4 =28
----------5 =35
----------6 =42
----------7 =49
----------8 =56
----------9 =63